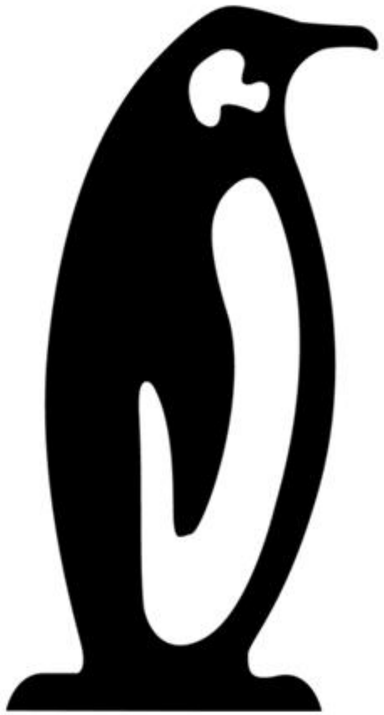


Linux Plumbers Conference

Vienna, Austria | September 18-20, 2024

Dynamic Update of CPU Capacity based on Steal Time

Shrikanth Hegde - IBM <sshegde@linux.ibm.com>
Srikar Donamraju - IBM <srikar@linux.ibm.com>



LINUX PLUMBERS CONFERENCE | Vienna, Austria
Sept. 18-20, 2024

Agenda



- Problem statement and experimental results
- Current Approaches
- Using CPU capacity as the framework
- Steal time and adjusting CPU capacity
- Issue in the load balancer
- Closing remarks

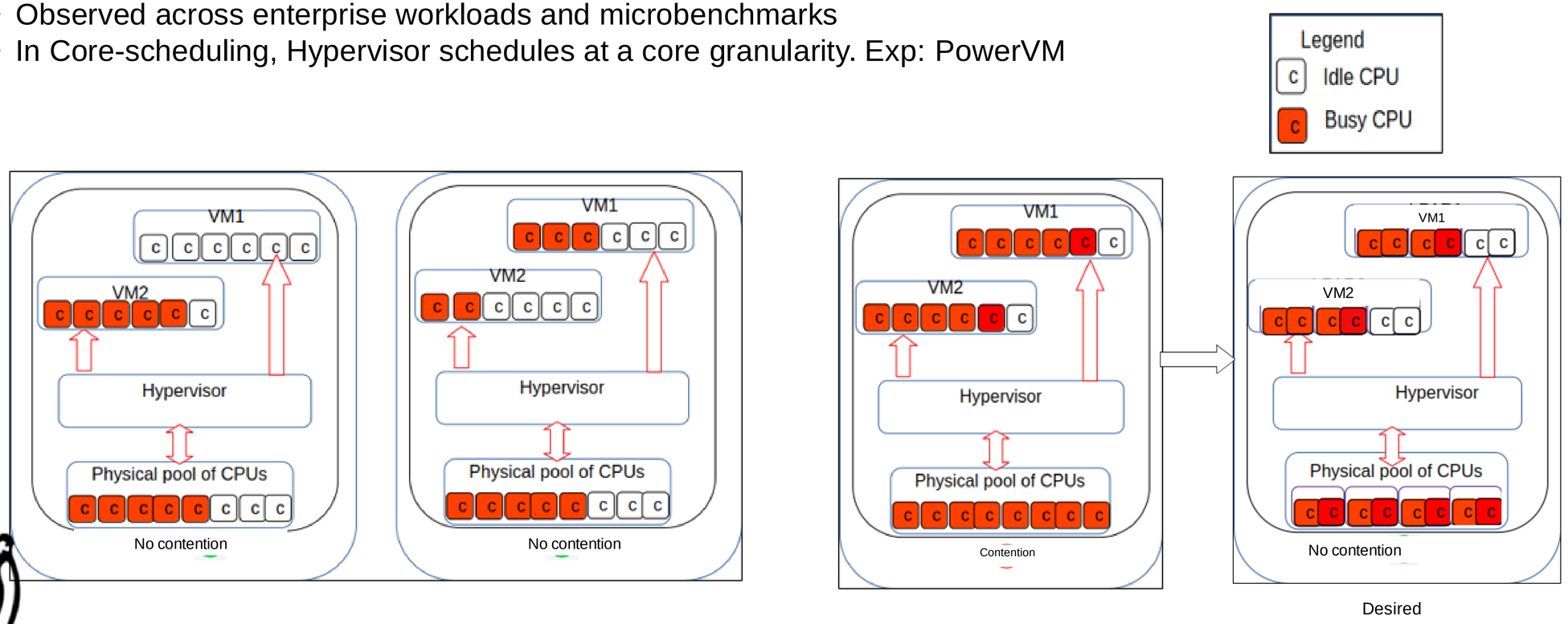


vCPU Preemption leads to performance impact

In a virtual environment where physical CPU(pCPU) is shared among different logical partitions (VM aka LPAR) and are overcommitting the pCPU i.e sum of vCPU > pCPU, when all the VMs have high utilization, it leads to frequent virtual CPU(vCPU) preemption.

In such Overcommitted and highly utilized VMs (Sum of vCPUs > CPUs)

- VM performance drops when there is contention for physical CPUs
- Observed across enterprise workloads and microbenchmarks
- In Core-scheduling, Hypervisor schedules at a core granularity. Exp: PowerVM



Experiments and Micro-benchmark results

VM	vCPUS	Entitled CPUS
1	94*8=752	47*8=376
2	94*8=752	47*8=376

Shared Physical Pool: 95*8 = 760

Hypervisor: PowerVM

- 2 VM/LPARs are running the same workload at the same time.
- Linux scheduler spreads the workload to cores.
- Taskset is used to illustrate the benefit of packing.

./stress-ng --cpu=376 --cpu-ops=10000000
successful run completed in **2 mins, 12.21 secs**
taskset -c 0-375 ./stress-ng --cpu=376 --cpu-ops=10000000
successful run completed in **1 min, 35.21 secs**

./hackbench 10 process 100000 loops
Time: 54.443
taskset -c 0-375 ./hackbench 10 process 100000 loops
Time: 12.417 **4x better**

./schbench -t 100 -m 3 -r 30 -i 30
Wakeup Latencies percentiles (usec) runtime 30 (s) (616989 total samples)
50.0th: **17** (176612 samples)
90.0th: **773** (234469 samples)
* 99.0th: **9680** (55527 samples)

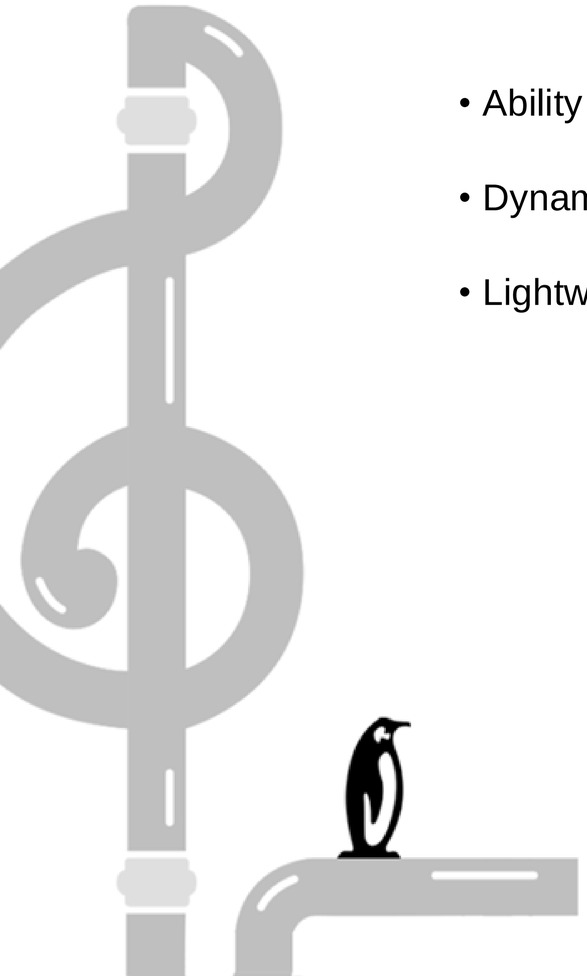
taskset -c 0-375 ./schbench -t 30 -m 10 -r 30 -i 30
Wakeup Latencies percentiles (usec) runtime 30 (s) (679027 total samples)
50.0th: **14** (255666 samples)
90.0th: **16** (211470 samples)
* 99.0th: **33** (54870 samples) - **Good Improvement**

Design Requirements

- Ability to run on all CPUs under low contention
- Dynamically Limit only under high contention
- Lightweight kernel Mechanism

Current Approaches

- **CPU Hotplug:**
  - Can enforce hard limits. Nothing runs on offlined CPUs.
  - Expensive and doesn't scale well.
  - Needs a daemon to monitor and hotplug CPUs as necessary.
- **Cpuset cgroup**:
  - More efficient
  - Needs a daemon to monitor and adjust cpusets accordingly.
  - Update of cgroup environment can break.
  - user can break with taskset/numactl
- **Asym Packing**:
  - Would be complicated to implement
  - Dynamic update needs a sched domain rebuild.



Can CPU Capacity be the right framework?

CPU capacity can be changed at runtime and it is not expensive to change it. It is expected that process's move when there is not enough capacity.

What if CPU capacity is changed to 1?

```
unsigned long arch_scale_cpu_capacity(int cpu)
{
    if (is_shared_processor() && cpu >= num_cpus_use)
        return 1;

    return SCHED_CAPACITY_SCALE;
}
```

```
static int pv_vp_manual_hint_set(void *data, u64 val)
{
        if (val <= 0)
                num_cpus_use = num_present_cpus();

        if (val != num_cpus_use) {
                num_cpus_use = val;
                powerpc_topology_update = 1;
                rebuild_sched_domains();
        }
        return 0;
}
```

Note: the `rebuild_sched_domains` is for initial experiments only. It is expensive and expect the CPU capacity changes to work without it.



Using Steal Time

Time spent in involuntary wait by the virtual CPU while the hypervisor was servicing another virtual CPU or hypervisor itself.

If steal time is accurate, then use that as the hint. Ideal would be H/W hint provided by the hypervisor.

```
if (low_steal)
        num_cpus_use = min(num_cpus_use + threads_per_core, num_present_cpus());
else
        num_cpus_use= max(num_cpus_use - threads_per_core, 1);
```

Which CPUs to select is a secondary problem once we have the right framework to limit to set of CPUs. This could be dependent on architecture.

Advantage we get is,

- arch can decide based on either steal time, or H/W based hit or through timer and set the CPU capacity accordingly
- arch can decide which of those CPUs as well. Doing that in the generic scheduler will be very tricky.



Capacity change verification

Manual hint for capacity was set at 376. It was verified by adding a debug fs file. This was done for experimentation and validation.

#lscpu

```
Architecture:            ppc64le
  Byte Order:            Little Endian
CPU(s):                  752
  On-line CPU(s) list:   0-751
Model name:              POWER10
```

/sys/devices/system/cpu/cpu* # cat cpu_capacity

```
cpu: 373, capacity: 1024
cpu: 374, capacity: 1024
cpu: 375, capacity: 1024
cpu: 376, capacity: 1
cpu: 377, capacity: 1
```

cat domain*/flags

SMT - SD_BALANCE_NEWIDLE SD_BALANCE_EXEC SD_BALANCE_FORK SD_SHARE_CPUCAPACITY SD_SHARE_LLC SD_WAKE_AFFINE
MC - SD_BALANCE_NEWIDLE
SD_BALANCE_EXEC SD_ASYM_PACKING SD_BALANCE_FORK SD_WAKE_AFFINE **SD_ASYM_CPUCAPACITY SD_ASYM_CPUCAPACITY_FULL**
PKG - SD_BALANCE_NEWIDLE **SD_ASYM_CPUCAPACITY SD_ASYM_CPUCAPACITY_FULL** SD_SERIALIZE SD_OVERLAP SD_NUMA

Experiments: Running schbench

Upstream Kernel, all CPU capacity = 1024

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	366	17.07	0.00	0.03	0.00	0.06	0.00	12.45	0.00	0.00	70.39
Average:	367	20.62	0.00	0.03	0.00	0.06	0.00	14.63	0.00	0.00	64.66
Average:	368	19.05	0.00	0.09	0.00	0.06	0.00	14.03	0.00	0.00	66.77
Average:	369	19.82	0.00	0.09	0.00	0.09	0.00	14.80	0.00	0.00	65.20
Average:	370	18.79	0.00	0.06	0.00	0.18	0.00	14.45	0.00	0.00	66.52
Average:	371	18.12	0.00	0.21	0.00	0.00	0.00	13.20	0.00	0.00	68.48
Average:	372	17.35	0.00	0.15	0.00	0.09	0.00	12.91	0.00	0.00	69.51
Average:	373	16.97	0.00	0.09	0.00	0.06	0.00	13.16	0.00	0.00	69.72
Average:	374	18.07	0.00	0.06	0.00	0.06	0.00	13.70	0.00	0.00	68.10
Average:	375	17.50	0.00	0.12	0.00	0.06	0.00	13.34	0.00	0.00	68.97
Average:	376	20.84	0.00	0.15	0.00	0.06	0.00	13.75	0.00	0.00	65.21
Average:	377	17.73	0.00	0.18	0.00	0.18	0.00	12.20	0.00	0.00	69.71
Average:	378	17.63	0.00	0.12	0.00	0.21	0.00	12.96	0.00	0.00	69.09
Average:	379	19.21	0.00	0.06	0.00	0.09	0.00	12.31	0.00	0.00	68.32
Average:	380	15.47	0.00	0.03	0.00	0.03	0.00	11.17	0.00	0.00	73.30
Average:	381	19.39	0.00	0.03	0.00	0.03	0.00	12.25	0.00	0.00	68.29
Average:	382	15.01	0.00	0.06	0.00	0.03	0.00	10.94	0.00	0.00	73.97
Average:	383	16.00	0.00	0.09	0.00	0.21	0.00	12.47	0.00	0.00	71.24
Average:	384	23.17	0.00	0.12	0.00	0.09	0.00	15.31	0.00	0.00	61.32
Average:	385	21.21	0.00	0.09	0.00	0.03	0.00	12.66	0.00	0.00	66.01
Average:	386	20.43	0.00	0.09	0.00	0.18	0.00	12.43	0.00	0.00	66.87
Average:	387	20.83	0.00	0.09	0.00	0.06	0.00	12.88	0.00	0.00	66.14
Average:	388	20.36	0.00	0.06	0.00	0.12	0.00	12.74	0.00	0.00	66.73
Average:	389	18.97	0.00	0.00	0.00	0.18	0.00	11.09	0.00	0.00	69.76
Average:	390	19.13	0.00	0.15	0.00	0.00	0.00	11.40	0.00	0.00	69.31
Average:	391	20.31	0.00	0.06	0.00	0.00	0.00	12.31	0.00	0.00	67.32
Average:	392	20.41	0.00	0.12	0.00	0.06	0.00	16.24	0.00	0.00	63.17
Average:	393	17.03	0.00	0.15	0.00	0.09	0.00	13.78	0.00	0.00	68.95
Average:	394	21.99	0.00	0.03	0.00	0.09	0.00	13.94	0.00	0.00	63.96
Average:	395	14.97	0.00	0.12	0.00	0.09	0.00	11.72	0.00	0.00	73.10
Average:	396	16.96	0.00	0.09	0.00	0.24	0.00	14.21	0.00	0.00	68.51

CPU Capacity = 1 if cpu > 376

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	366	87.09	0.00	0.00	0.00	0.21	0.00	0.03	0.00	0.00	12.67
Average:	367	86.93	0.00	0.03	0.00	0.38	0.00	0.03	0.00	0.00	12.63
Average:	368	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	369	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	370	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	371	0.00	0.00	0.00	0.00	0.00	0.00	0.03	0.00	0.00	99.97
Average:	372	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	373	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	374	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	375	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	376	82.15	0.00	0.03	0.00	1.17	0.00	0.03	0.00	0.00	16.63
Average:	377	82.18	0.00	0.03	0.00	0.75	0.00	0.03	0.00	0.00	17.01
Average:	378	82.12	0.00	0.12	0.00	0.75	0.00	0.00	0.00	0.00	17.02
Average:	379	82.21	0.00	0.00	0.00	0.80	0.00	0.00	0.00	0.00	16.98
Average:	380	81.71	0.00	0.03	0.00	0.89	0.00	0.03	0.00	0.00	17.34
Average:	381	81.44	0.00	0.15	0.00	0.81	0.00	0.00	0.00	0.00	17.61
Average:	382	81.48	0.00	0.00	0.00	0.83	0.00	0.00	0.00	0.00	17.68
Average:	383	80.97	0.00	0.06	0.00	1.12	0.00	0.00	0.00	0.00	17.84
Average:	384	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	385	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	386	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00

CPU Capacity = 1 if cpu > 376 and Load balancer changes

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	366	89.53	0.00	0.03	0.00	0.15	0.00	0.03	0.00	0.00	10.26
Average:	367	89.71	0.00	0.03	0.00	0.06	0.00	0.00	0.00	0.00	10.19
Average:	368	89.61	0.00	0.00	0.00	0.06	0.00	0.03	0.00	0.00	10.29
Average:	369	89.46	0.00	0.03	0.00	0.15	0.00	0.03	0.00	0.00	10.33
Average:	370	89.44	0.00	0.06	0.00	0.12	0.00	0.03	0.00	0.00	10.34
Average:	371	89.48	0.00	0.06	0.00	0.06	0.00	0.03	0.00	0.00	10.37
Average:	372	89.58	0.00	0.06	0.00	0.03	0.00	0.03	0.00	0.00	10.30
Average:	373	89.45	0.00	0.06	0.00	0.09	0.00	0.03	0.00	0.00	10.36
Average:	374	89.54	0.00	0.00	0.00	0.12	0.00	0.03	0.00	0.00	10.31
Average:	375	89.44	0.00	0.06	0.00	0.15	0.00	0.03	0.00	0.00	10.32
Average:	376	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	377	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	378	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	379	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	380	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	381	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	382	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	383	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00





Issues observed in load balancer

1. In `update_sd_pick_busiest`, asym capacity works only if local group type is group_has_spare. Else It will skip that group. In this case, it needs to be pull always if there is a task on very low capacity CPU

2. `Task_fits_cpu` – returns 1 sometimes when CPU capacity set to 1. This happens because util=0 though the rq had task running.

3. Is `group_misfit_task` is designed to honor CPU capacity and move tasks, the `group_smt_balance` might interfere with it, if the load balancing is happening between two cores i.e at sched_domain is at PKG/MC level.

4. Need to make group_misfit_task as the highest priority to handle high concurrency case.

5. Setting misfit, in `update_misfit_status`, doesn't set misfit task load even when capacity is 1. Again due to `task_fits_cpu`

6. `check_cpu_capacity` – this checks if capacity has been reduced due to DL or RT or IRQ load. Since that's not the case here, it leads to 0. Hence not triggering the active load balance.

What are the possible approaches?

- Introduce a new group type.
- Continue using misfit and fix it for this case and ensure it doesn't break regular asym capacity case.



CPU capacity as the framework

Pros:

Capacity Can be changed at runtime.
Not-expensive. (lightweight)
Architecture hooks are present already. Arch can set capacity based on its hints.
Arch can also decide which CPUs in its own way
Doesn’t need a sched domain rebuilding.
Behavior can be altered with load balancer.

Cons:

Honored by sched_normal tasks only. Irq or kthreads can still run on that low capacity CPU.
Deadline and Realtime may not honor the capacity.
Packing is best effort. Under high concurrency, task may shift to low capacity CPUs.



References:

Powerpc shared lpar asym packing: https://lore.kernel.org/lkml/20231214180720.310852-1-srikar@linux.vnet.ibm.com/
X86 asym packing: https://lore.kernel.org/all/20230406203148.19182-1-ricardo.neri-calderon@linux.intel.com/
Group_smt_balance: https://lore.kernel.org/all/cover.1688770494.git.tim.c.chen@linux.intel.com/

CPU Hotplug Improvement efforts:
https://lore.kernel.org/all/Y01UWQL2y2r69sBX@li-05afa54c-330e-11b2-a85c-e3f3aa0db1e9.ibm.com/#r
https://lore.kernel.org/all/5f2ab8a44d685701fe36cdaa8042a1aef215d10d.camel@linux.vnet.ibm.com/#r

Steal time Related patches in powerpc:
https://lore.kernel.org/all/20240213052635.231597-1-sshegde@linux.ibm.com/
https://lore.kernel.org/all/20220518133935.3878954-4-npiggin@gmail.com/#r





Legal Statement

- This work represents the view of the author and does not necessarily represent the view of IBM.
- IBM and IBM(logo) are trademarks or registered trademarks of International Business Machines Corporation in the United States and/or other countries.
- Linux is a registered trademark of Linus Torvalds
- Other company, product, and service names may be trademarks or service marks of others.



© 2023 IBM Corporation

Thank you



Results with manual hint.

Microbenchmark results with setting CPU capacity =1 if CPU > 375 with load balancer changes.
Numbers are similar to taskset experiments.

./stress-ng --cpu=375 --cpu-ops=10000000
stress-ng: info: [21804] successful run completed in 1 min, 31.21 secs

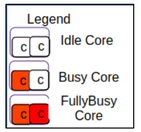
./hackbench 20 process 10000 loops
Running with 20*40 (== 800) tasks.
Time: 12.306

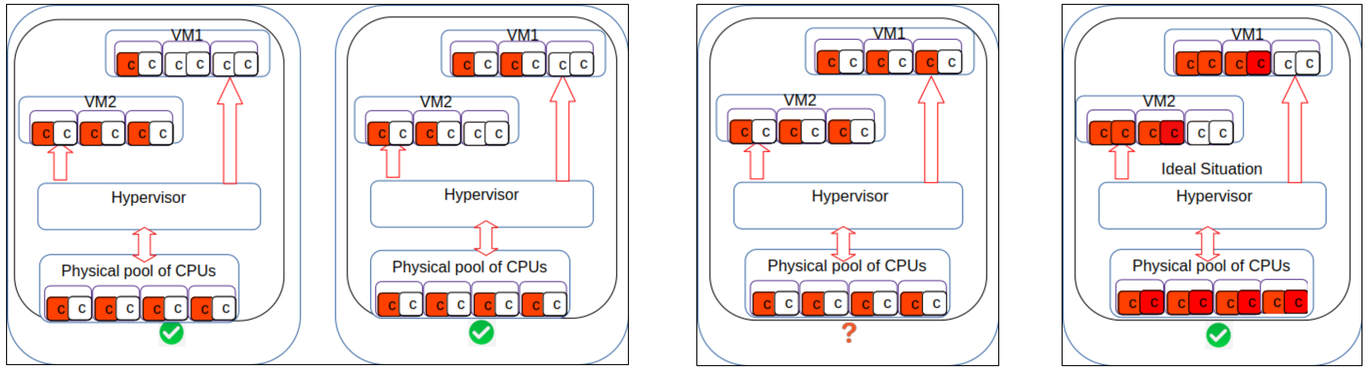
./schbench -t 30 -m 10 -r 30 -i 30
Wakeup Latencies percentiles (usec) runtime 30 (s) (682822 total samples)
50.0th: 25 (214089 samples)
90.0th: 35 (240763 samples)
* 99.0th: 67 (60810 samples)



Core scheduling Hypervisor

- In Core-scheduling, Hypervisor schedules at a core granularity.
- PowerVM Hypervisor implements Core-scheduling.
- Also PowerVM Hypervisor by default does vCPU pinning to a logical Core.





Trace_printks

```
 <idle>-0       [018] ..s..   482.689798: update_sd_lb_stats.constprop.158: misfit task load is present, but smt_balance is set at name: PKG
  <idle>-0      [018] ..s..   482.689831: update_sd_lb_stats.constprop.158: misfit task load is present, but smt_balance is set at name: PKG
  <idle>-0      [018] ..s..   482.689832: update_sd_lb_stats.constprop.158: misfit task load is present, but smt_balance is set at name: PKG
  <idle>-0      [018] ..s..   482.689833: update_sd_lb_stats.constprop.158: misfit task load is present, but smt_balance is set at name: PKG
  <idle>-0      [018] ..s..   482.689834: update_sd_lb_stats.constprop.158: misfit task load is present, but smt_balance is set at name: PKG
```

```
   stress-ng-cpu-8637    [655] d.h..    61.474646: util_fits_cpu.isra.111: cpu: 655, fits: 0, util: 37, capacity: 1, uclamp_used: 0 task_name: stress-ng-cpu, task_load
: 1023
   stress-ng-cpu-8756    [651] d.h..    61.474646: util_fits_cpu.isra.111: cpu: 651, fits: 0, util: 25, capacity: 1, uclamp_used: 0 task_name: stress-ng-cpu, task_load
: 996
```

```
stress-ng-cpu-8899    [392] dnh..     util_fits_cpu.isra.111: cpu: 392, fits: 1, util: 0, capacity: 1, uclamp_used: 0 task_name: stress-ng-cpu, task_load: 1023
stress-ng-cpu-8921    [394] dnh..     util_fits_cpu.isra.111: cpu: 394, fits: 1, util: 0, capacity: 1, uclamp_used: 0 task_name: stress-ng-cpu, task_load: 1023

stress-ng-cpu-8925    [394] dnh..    61.444651: task_tick_fair: Misfit is not applicable: cpu: 394, p_nr_cpus_allowed:752, p_max_capacity: 1024, task_fit_cpu: 1,
arc_cpu_cap: 1
```



Upstream Kernel, all CPU capacity = 1024

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	360	21.11	0.00	0.12	0.00	0.21	0.00	14.00	0.00	0.00	64.56
Average:	361	22.63	0.00	0.15	0.00	0.24	0.00	16.11	0.00	0.00	60.88
Average:	362	17.72	0.00	0.24	0.00	0.09	0.00	12.72	0.00	0.00	69.23
Average:	363	21.51	0.00	0.12	0.00	0.18	0.00	14.71	0.00	0.00	63.48
Average:	364	15.96	0.00	0.03	0.00	0.06	0.00	11.50	0.00	0.00	72.45
Average:	365	20.15	0.00	0.03	0.00	0.12	0.00	15.19	0.00	0.00	64.52
Average:	366	17.07	0.00	0.03	0.00	0.06	0.00	12.45	0.00	0.00	70.39
Average:	367	20.62	0.00	0.03	0.00	0.06	0.00	14.63	0.00	0.00	64.66
Average:	368	19.05	0.00	0.09	0.00	0.06	0.00	14.03	0.00	0.00	66.77
Average:	369	19.82	0.00	0.09	0.00	0.09	0.00	14.80	0.00	0.00	65.20
Average:	370	18.79	0.00	0.06	0.00	0.18	0.00	14.45	0.00	0.00	66.52
Average:	371	18.12	0.00	0.21	0.00	0.00	0.00	13.20	0.00	0.00	68.48
Average:	372	17.35	0.00	0.15	0.00	0.09	0.00	12.91	0.00	0.00	69.51
Average:	373	16.97	0.00	0.09	0.00	0.06	0.00	13.16	0.00	0.00	69.72
Average:	374	18.07	0.00	0.06	0.00	0.06	0.00	13.70	0.00	0.00	68.10
Average:	375	17.50	0.00	0.12	0.00	0.06	0.00	13.34	0.00	0.00	68.97
Average:	376	20.84	0.00	0.15	0.00	0.06	0.00	13.75	0.00	0.00	65.21
Average:	377	17.73	0.00	0.18	0.00	0.18	0.00	12.20	0.00	0.00	69.71
Average:	378	17.63	0.00	0.12	0.00	0.21	0.00	12.96	0.00	0.00	69.09
Average:	379	19.21	0.00	0.06	0.00	0.09	0.00	12.31	0.00	0.00	68.32
Average:	380	15.47	0.00	0.03	0.00	0.03	0.00	11.17	0.00	0.00	73.30
Average:	381	19.39	0.00	0.03	0.00	0.03	0.00	12.25	0.00	0.00	68.29
Average:	382	15.01	0.00	0.06	0.00	0.03	0.00	10.94	0.00	0.00	73.97
Average:	383	16.00	0.00	0.09	0.00	0.21	0.00	12.47	0.00	0.00	71.24
Average:	384	23.17	0.00	0.12	0.00	0.09	0.00	15.31	0.00	0.00	61.32
Average:	385	21.21	0.00	0.09	0.00	0.03	0.00	12.66	0.00	0.00	66.01
Average:	386	20.43	0.00	0.09	0.00	0.18	0.00	12.43	0.00	0.00	66.87
Average:	387	20.83	0.00	0.09	0.00	0.06	0.00	12.88	0.00	0.00	66.14
Average:	388	20.36	0.00	0.06	0.00	0.12	0.00	12.74	0.00	0.00	66.73
Average:	389	18.97	0.00	0.00	0.00	0.18	0.00	11.09	0.00	0.00	69.76
Average:	390	19.13	0.00	0.15	0.00	0.00	0.00	11.40	0.00	0.00	69.31



CPU Capacity = 1 if CPU > 376

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	360	87.11	0.00	0.00	0.00	0.24	0.00	0.00	0.00	0.00	12.65
Average:	361	87.23	0.00	0.03	0.00	0.15	0.00	0.00	0.00	0.00	12.60
Average:	362	86.86	0.00	0.00	0.00	0.44	0.00	0.00	0.00	0.00	12.69
Average:	363	86.97	0.00	0.00	0.00	0.38	0.00	0.00	0.00	0.00	12.64
Average:	364	87.11	0.00	0.03	0.00	0.18	0.00	0.03	0.00	0.00	12.66
Average:	365	86.97	0.00	0.09	0.00	0.32	0.00	0.00	0.00	0.00	12.61
Average:	366	87.09	0.00	0.00	0.00	0.21	0.00	0.03	0.00	0.00	12.67
Average:	367	86.93	0.00	0.03	0.00	0.38	0.00	0.03	0.00	0.00	12.63
Average:	368	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	369	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	370	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	371	0.00	0.00	0.00	0.00	0.00	0.00	0.03	0.00	0.00	99.97
Average:	372	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	373	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	374	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	375	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	376	82.15	0.00	0.03	0.00	1.17	0.00	0.03	0.00	0.00	16.63
Average:	377	82.18	0.00	0.03	0.00	0.75	0.00	0.03	0.00	0.00	17.01
Average:	378	82.12	0.00	0.12	0.00	0.75	0.00	0.00	0.00	0.00	17.02
Average:	379	82.21	0.00	0.00	0.00	0.80	0.00	0.00	0.00	0.00	16.98
Average:	380	81.71	0.00	0.03	0.00	0.89	0.00	0.03	0.00	0.00	17.34
Average:	381	81.44	0.00	0.15	0.00	0.81	0.00	0.00	0.00	0.00	17.61
Average:	382	81.48	0.00	0.00	0.00	0.83	0.00	0.00	0.00	0.00	17.68
Average:	383	80.97	0.00	0.06	0.00	1.12	0.00	0.00	0.00	0.00	17.84
Average:	384	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	385	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	386	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	387	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	388	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	389	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	390	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00



CPU Capacity = 1 if CPU > 376 and Load balancer changes

Average:	CPU	%usr	%nice	%sys	%iowait	%irq	%soft	%steal	%guest	%gnice	%idle
Average:	360	89.63	0.00	0.03	0.00	0.06	0.00	0.03	0.00	0.00	10.25
Average:	361	86.95	0.00	0.15	0.00	0.06	0.00	0.00	0.00	0.00	12.84
Average:	362	89.66	0.00	0.06	0.00	0.09	0.00	0.00	0.00	0.00	10.18
Average:	363	71.48	0.00	0.42	0.00	0.12	0.00	0.00	0.00	0.00	27.98
Average:	364	69.68	0.00	0.09	0.00	0.15	0.00	0.03	0.00	0.00	30.05
Average:	365	89.52	0.00	0.06	0.00	0.15	0.00	0.03	0.00	0.00	10.23
Average:	366	89.53	0.00	0.03	0.00	0.15	0.00	0.03	0.00	0.00	10.26
Average:	367	89.71	0.00	0.03	0.00	0.06	0.00	0.00	0.00	0.00	10.19
Average:	368	89.61	0.00	0.00	0.00	0.06	0.00	0.03	0.00	0.00	10.29
Average:	369	89.46	0.00	0.03	0.00	0.15	0.00	0.03	0.00	0.00	10.33
Average:	370	89.44	0.00	0.06	0.00	0.12	0.00	0.03	0.00	0.00	10.34
Average:	371	89.48	0.00	0.06	0.00	0.06	0.00	0.03	0.00	0.00	10.37
Average:	372	89.58	0.00	0.06	0.00	0.03	0.00	0.03	0.00	0.00	10.30
Average:	373	89.45	0.00	0.06	0.00	0.09	0.00	0.03	0.00	0.00	10.36
Average:	374	89.54	0.00	0.00	0.00	0.12	0.00	0.03	0.00	0.00	10.31
Average:	375	89.44	0.00	0.06	0.00	0.15	0.00	0.03	0.00	0.00	10.32
Average:	376	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	377	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	378	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	379	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	380	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	381	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	382	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	383	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	384	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	385	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	386	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	387	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	388	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	389	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00
Average:	390	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	100.00



Schbench results with capacity values
schbench -t 100 -m 3 -r 30 -i 30

Upstream

```
.Wakeup Latencies percentiles (usec)
runtime 30 (s) (616989 total samples)
          50.0th: 17         (176612 samples)
          90.0th: 773        (234469 samples)
        * 99.0th: 9680       (55527 samples)
```

Upstream with taskset

```
taskset -c 0-375 ./schbench -t 100 -m 3 -r 30 -i 30
Wakeup Latencies percentiles (usec) runtime 30
(s) (737371 total samples)
          50.0th: 14         (268397 samples)
          90.0th: 16         (224926 samples)
        * 99.0th: 33         (56894 samples)
```

CPU Capacity = 1 if CPU > 376

```
Wakeup Latencies percentiles (usec) runtime
30 (s) (612524 total samples)
          50.0th: 19         (173221 samples)
          90.0th: 65         (238694 samples)
        * 99.0th: 787        (54572 samples)
```

CPU Capacity = 1 if CPU > 376 and Load balancer changes

```
Wakeup Latencies percentiles (usec) runtime
30 (s) (679027 total samples)
          50.0th: 20         (255666 samples)
          90.0th: 28         (211470 samples)
        * 99.0th: 53         (54870 samples)
```



Linux Plumbers Conference

Vienna, Austria | September 18-20, 2024